SECTION A — CONCEPT APPLICATION

S1

QUESTION

You are building a delivery fee calculator for a food delivery platform. You have a NumPy array of 50,000 order distances (in km) and need to compute a delivery fee for every order simultaneously. A teammate suggests iterating over the array using a Python for-loop and appending results to a list.

Explain why NumPy vectorisation is a technically superior approach to the for-loop suggestion. In your answer, describe what broadcasting allows when a single scalar fee rate (e.g., Rs 5 per km) is applied to the entire array, and identify one scenario where a Python loop would still be necessary despite NumPy being available.

ANSWER

NumPy vectorisation is technically superior to a Python for-loop because it performs operations on the entire NumPy array at once instead of processing each order individually. It reduces Python-level loop overhead and is generally faster and more efficient when working with a large number of numerical values such as 50,000 delivery distances.

Broadcasting allows a single scalar value, such as Rs 5 per kilometre, to be automatically applied to every element of the NumPy array without explicitly writing a loop. For example:

fees = distances * 5

If the distances are `[2, 5, 10]`, NumPy automatically applies `5` to each value and produces `[10, 25, 50]`.

A Python loop would still be necessary when each order requires complex individual processing that cannot be conveniently expressed using NumPy operations, such as making a separate external API request for each order.

S2

QUESTION

You are working on a reporting pipeline for a food delivery platform. You have a Pandas DataFrame with columns: `order_id`, `restaurant_name`, `city`, `order_value`,

`delivery_time_mins`, and `rating`. A colleague proposes filtering the DataFrame in a for-loop once per restaurant and computing averages manually inside the loop.

Explain why `groupby()` combined with `agg()` is the correct approach instead of the for-loop strategy. Describe the specific method chain you would write to compute average order value, average delivery time, and average rating per restaurant in a single operation, and explain what the resulting DataFrame's index represents after the operation.

ANSWER

`groupby()` combined with `agg()` is the correct approach because it allows the DataFrame to be divided into groups based on `restaurant_name` and multiple summary calculations to be performed on every group in a single operation. This avoids manually filtering and calculating averages for each restaurant using a Python for-loop.

The method chain would be:

```
restaurant_summary = (
    df.groupby("restaurant_name")
      .agg(
          mean_order_value=("order_value", "mean"),
          mean_delivery_time=("delivery_time_mins", "mean"),
          mean_rating=("rating", "mean")
      )
)
```

This calculates the average order value, average delivery time, and average rating for each restaurant.

After the operation, the resulting DataFrame's index represents the unique `restaurant_name` values because `restaurant_name` is used as the grouping column.

S3

QUESTION

You are developing a monthly performance report for a food delivery startup. The report must present three pieces of information side by side:
(a) total orders placed per month over a year,
(b) the proportion of orders by cuisine type (Fast Food, Indian, Chinese, Desserts), and
(c) the distribution of delivery times across all orders.

Identify the most appropriate Matplotlib chart type for each of the three metrics and justify each choice based on the nature of the data. Then explain how `plt.subplots()`

would be configured to arrange all three charts in a single figure, and state whether a shared x-axis would or would not be appropriate for this layout — and why.

ANSWER

1. **Total orders per month — Line Chart**

A line chart is appropriate because the data represents the number of orders over 12 months. It shows changes and trends in order volume over time clearly.

2. **Proportion of orders by cuisine type — Pie Chart**

A pie chart is appropriate because the data represents the proportion of total orders belonging to different cuisine categories. It allows the contribution of Fast Food, Indian, Chinese, and Desserts to be compared as parts of the whole.

3. **Distribution of delivery times — Histogram**

A histogram is appropriate because delivery time is numerical data. It shows how the delivery times are distributed across different time intervals.

The three charts can be arranged side by side using:

```
fig, axes = plt.subplots(1, 3, figsize=(18, 5))
```

This creates one row with three columns, allowing all three charts to appear in the same figure.

A shared x-axis would **not** be appropriate because the three charts have different types of x-axis data: months, cuisine categories, and delivery times. Therefore, each subplot should have its own x-axis.

S4

QUESTION

You are optimising a Matplotlib line chart that shows daily order volume for a food delivery app over a 90-day period. The chart renders correctly but stakeholders keep asking "which day had the spike?" during every review meeting. The spike is the single highest-value day in the array.

Describe how you would use `plt.annotate()` to programmatically identify and mark the maximum order day with an arrow pointing to the peak and a text label showing the day number and volume. Name at least two specific parameters you would pass to `plt.annotate()` and explain what each one controls in the rendered chart.

ANSWER

First, the maximum order volume can be identified programmatically using `np.argmax()`:

```
max_index = np.argmax(order_volume)
max_day = days[max_index]
max_volume = order_volume[max_index]
```

Then `plt.annotate()` can be used to mark the maximum point:

```
plt.annotate(
    f"Day {max_day}: {max_volume}",
    xy=(max_day, max_volume),
    xytext=(max_day, max_volume + 100),
    arrowprops=dict(arrowstyle="->")
)
```

The important parameters are:

- **`xy`** — specifies the coordinates of the point that should be annotated. Here, it identifies the maximum order day and its order volume.
- **`xytext`** — specifies the position where the annotation text will be displayed.
- **`arrowprops`** — controls the appearance and style of the arrow pointing toward the maximum point.

This allows the program to automatically identify the highest order day and display both the day number and order volume on the chart.

S5

QUESTION

You are designing a visual analysis of customer ratings for a food delivery app. Your dataset has 10,000 rows. Each row belongs to one of three restaurant categories (Fast Food, Fine Dining, Casual) and has a customer rating on a discrete 1–5 scale. Your manager wants to understand how ratings are distributed within each category and whether any category produces more extreme outliers.

Compare `sns.boxplot()` and `sns.violinplot()` for this specific task. Which would you recommend and why? Provide at least two data-specific reasons that reference properties of this dataset (10,000 rows, 3 categories, discrete 1–5 scale), and explain one situation where the other chart type would be preferable.

ANSWER

For this dataset, I would recommend **`sns.boxplot()`**.

First, the dataset contains **10,000 rows**, so a boxplot can provide a clear summary of the large number of observations in each restaurant category without displaying every individual data point. It shows the median, quartiles, spread, and potential outliers.

Second, there are only **three categories** — Fast Food, Fine Dining, and Casual — so the distributions can be easily compared side by side using a boxplot.

The ratings are also on a **discrete 1–5 scale**. A boxplot provides a simple and clear summary of these rating values and makes extreme outliers easier to identify, which is specifically required by the manager.

A **violin plot** would be preferable when the main objective is to examine the detailed shape and density of a distribution, especially for a continuous variable with many possible values.

S6

QUESTION

You are building a feature analysis report for a food delivery platform's data science team before a regression model is trained. Your DataFrame contains eight numeric columns: `order_value`, `distance_km`, `delivery_time_mins`, `rating`, `discount_applied`, `peak_hour_flag`, `restaurant_age_years`, and `driver_experience_years`.

Explain how you would generate and interpret a Seaborn correlation heatmap to identify multicollinearity risks among these features. In your answer, describe what the colour scale in the heatmap represents, how to read a specific cell value, and what correlation magnitude threshold you would use to flag a potential multicollinearity problem — and why that threshold is conventionally chosen.

ANSWER

First, I would select the eight numeric columns and calculate their Pearson correlation matrix using Pandas:

```
corr = df[
    [
        "order_value",
        "distance_km",
        "delivery_time_mins",
        "rating",
        "discount_applied",
        "peak_hour_flag",
        "restaurant_age_years",
```

```
        "driver_experience_years"
    ]
].corr()
```

Then I would create a Seaborn heatmap:

```
sns.heatmap(corr, annot=True, cmap="coolwarm")
plt.show()
```

The **colour scale** represents the strength and direction of the correlation between two variables. A correlation close to **+1** indicates a strong positive relationship, a correlation close to **-1** indicates a strong negative relationship, and a correlation close to **0** indicates little or no linear relationship.

For example, if the cell between `distance_km` and `delivery_time_mins` contains **0.85**, it indicates a strong positive correlation between these two features. As one increases, the other tends to increase as well.

A practical threshold of **|correlation| ≥ 0.8** can be used to flag a potential multicollinearity problem. This threshold is commonly used as a rule of thumb because a correlation of this magnitude indicates a strong linear relationship and suggests that two predictor variables may contain substantial overlapping information. However, it is a guideline rather than an absolute statistical cutoff.